

Components of Operating System

1. Kernel

The **kernel** is the main part of the operating system that talks directly to the **hardware** (CPU, RAM, Disk, Devices).

It performs all the important and low-level tasks.

Key Points

- **Heart of the OS** → Without kernel, OS cannot work.
- **First to load** when the computer starts (boot time).
- Controls CPU, memory, files, and devices.
- Has **full control** over the system (highest privilege).

Simple Example

When you open a file:

- App asks *Kernel*
- Kernel talks to *Hard Disk*
- File opens on screen

2. User Space

User space is the place where **applications run**.

Apps **cannot** directly touch hardware.

They must **ask the kernel** for any hardware-related work.

Examples of things in User Space

- **GUI (Graphical User Interface)**
 - Windows screen
 - Buttons, icons, apps
 - Example: YouTube app, Chrome, VS Code
- **CLI (Command Line Interface)**
 - Terminal
 - Commands like ls, cd, mkdir
 - Used to interact with OS using text

Example

You open Chrome → Chrome runs in **user space**

Chrome wants to read a file or use internet → It requests the **kernel**.

3.Shell

A **shell** is a program in the operating system that takes commands from the user and tells the OS to execute them.

It works like a **middleman** between the user and the OS.

Easy Explanation

- You type a command → Shell reads it
- Shell sends the request to the **kernel**
- Kernel performs the action
- Shell shows you the output

So, a shell is basically a **command interpreter**.

Example

If you type in terminal:

ls

- Shell receives the command
- Shell asks the kernel to list files
- Kernel returns the list
- Shell displays it on screen

Shells include:

- **Bash**
- **Zsh**
- **PowerShell**
- **Sh**

4.Functions of Kernel

1. Process Management

The kernel controls all processes and threads in the system.

- **Scheduling:** Decides which process or thread will run on the CPU.
- **Creating & deleting:** Starts new processes and removes finished ones.
- **Suspending & resuming:** Pauses a process and continues it later.
- **Synchronization & communication:** Helps processes talk to each other safely and work together.

2. Memory Management

The kernel handles all memory-related tasks.

- **Allocating & deallocating memory** to processes when required.
- **Tracking memory usage**: Keeps a record of which part of memory is used and by which process.

3. File Management

The kernel manages the entire file system.

- **Create and delete files**.
- **Create and delete directories** to organize files.
- **Map files to secondary storage** (hard disk/SSD).
- **Provide backup support** on stable storage.

4. I/O Management

The kernel manages input/output devices and their operations.

Important concepts

i. Spooling

Used when two devices work at **different speeds**.

Example:

- Print Spooling: Print jobs stored in a queue
- Mail Spooling

ii. Buffering

Data temporarily stored while transferring within **one job**.

Example:

- YouTube video buffering

iii. Caching

Storing frequently used data for **faster access**.

Example:

- Memory cache
- Web cache

5.Characteristics of Kernel

- All functions are inside the kernel.
- It is **large/bulky** in size.
- Needs **more memory** to run.
- **Less reliable** → if one module crashes, the whole kernel stops.
- **High performance** → user mode and kernel mode switching is fast.

Kernel Type	Meaning (Short)	Advantages	Disadvantages	Examples
Monolithic Kernel	All OS services (memory, process, file, I/O) run inside kernel space.	• Fast performance (direct communication) • Better hardware control	• Large size • Less secure (one crash → whole system crash)	Linux, Unix, MS-DOS
Microkernel	Only essential services run in kernel (rest run in user space).	• More secure • More stable (failure in user services doesn't crash kernel) • Easier to maintain	• Slower (more context switching) • Communication overhead	QNX, Minix, macOS (hybrid-based microkernel)
Hybrid Kernel	Mix of monolithic + microkernel.	• Better performance than microkernel • More stable than monolithic • Good balance	• Still complex • Not fully microkernel-level secure	Windows NT, modern macOS
Nanokernel	Extremely small kernel; minimal hardware abstraction only.	• Very small & fast • Best for embedded systems	• Very limited features • Needs more components outside kernel	Low-level embedded systems
Exokernel	Gives direct hardware access to applications; kernel only enforces security.	• Maximum flexibility • Very high performance	• Very complex for developers • Less portable	MIT Exokernel project

Feature	Monolithic Kernel	Microkernel
Speed	Fast	Slower
Size	Big (bulky)	Small & lightweight
Security	Less secure (all runs in kernel space)	More secure (most services in user space)
Stability	Less stable (one crash can crash whole system)	More stable (failure of services doesn't affect kernel)
Communication	Direct function calls → faster	Message passing → slower
Performance	High	Lower compared to monolithic
Example	Linux, Unix	Minix, QNX, macOS (microkernel-based)

6. How does communication happen between User Mode and Kernel Mode?

Ans. Through Inter-Process Communication (IPC).

Explanation

Even though **user processes** and the **kernel** run in separate memory spaces (for safety), they still need to communicate.

This communication happens using **IPC (Inter-Process Communication)**.

Two main IPC methods

- Shared Memory**
 - A common memory region is shared between processes.
 - Fast communication.
 - Example: Two apps reading/writing into the same shared buffer.
- Message Passing**
 - Processes send messages through the kernel.

- Safer but slower.
- Example: Sending a message/command to the kernel using system calls.